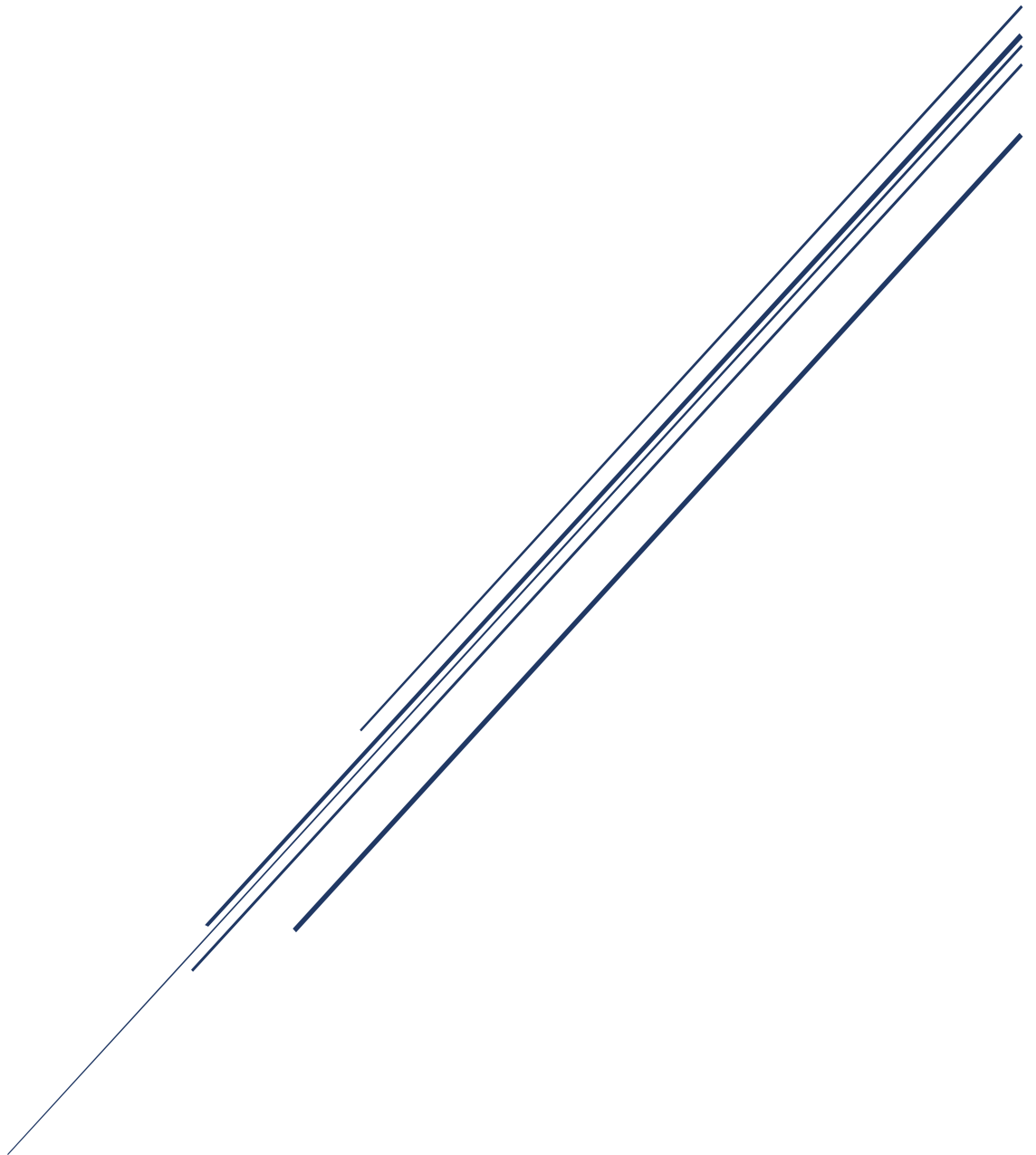


SAE3.02 : DEVELOPPEMENT DES APPLICATIONS COMMUNICANTES



**IUT de Colmar
2025 - 2026**

SOMMAIRE

Introduction

Architecture générale du projet

Installation du projet

Guide d'utilisation

Résultats attendus

Remarques importantes

Introduction

Ce document a pour objectif de décrire les étapes d'installation, de configuration et d'utilisation du projet SAE3.02.

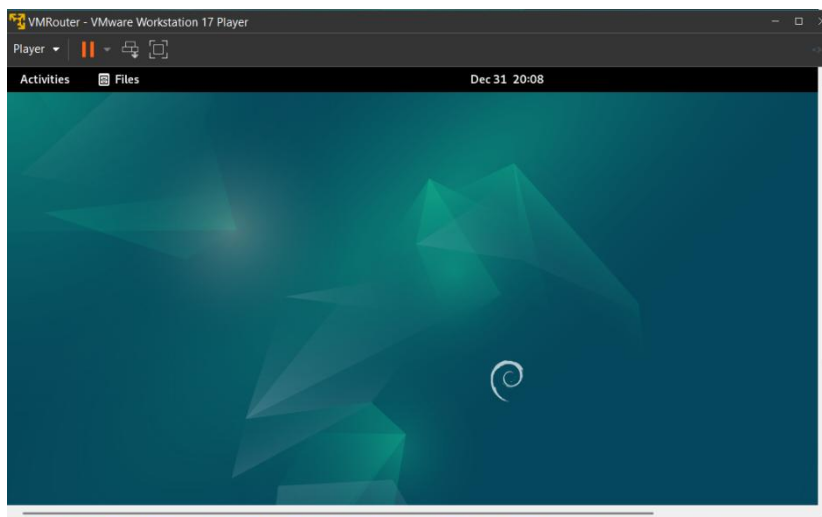
Il est destiné à toute personne souhaitant installer le projet sur son propre environnement, notamment l'enseignant évaluateur, afin de pouvoir tester le fonctionnement du réseau, observer les échanges chiffrés et valider les résultats obtenus.

Le projet repose sur une architecture client / routeurs / master, répartie sur plusieurs machines virtuelles connectées sur un même réseau.

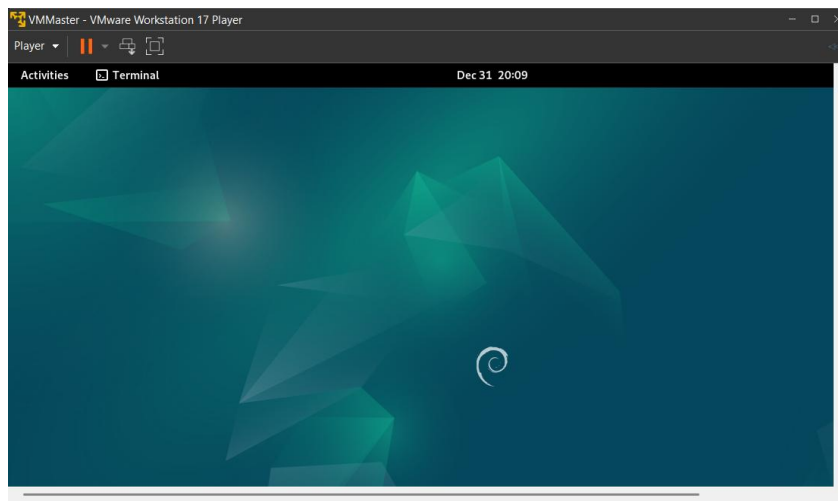
Architecture générale du projet

Le projet est déployé sur trois machines virtuelles distinctes :

- Une machine Master, chargée de la gestion des routeurs et de la distribution des clés publiques ;
- Une machine Routeurs, sur laquelle sont lancés plusieurs routeurs intermédiaires ;
- Une machine Clients, sur laquelle sont exécutées les interfaces graphiques Client A et Client B.



VMRouter



VMMaster



VMClient

Installation du projet

Prérequis système

Les machines virtuelles utilisées sont configurées comme suit :

- Système d'exploitation : **Debian 12** pour le Master et les Routeurs, **Windows** pour les Clients ;
- Mode réseau : **NAT**, permettant aux machines de communiquer entre elles ;
- Python version **3.10 ou supérieure**.

Avant toute installation, il est recommandé de vérifier la connectivité réseau entre les machines à l'aide de la commande suivante : ping IP_VM

Ping depuis la VMRouter

```
toto@Debian12:~$ ping 192.168.109.128
PING 192.168.109.128 (192.168.109.128) 56(84) bytes of data.
64 bytes from 192.168.109.128: icmp_seq=1 ttl=128 time=1.03 ms
64 bytes from 192.168.109.128: icmp_seq=2 ttl=128 time=1.72 ms
^C
--- 192.168.109.128 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.029/1.374/1.719/0.345 ms
toto@Debian12:~$ ping 192.168.109.129
PING 192.168.109.129 (192.168.109.129) 56(84) bytes of data.
64 bytes from 192.168.109.129: icmp_seq=1 ttl=64 time=0.534 ms
64 bytes from 192.168.109.129: icmp_seq=2 ttl=64 time=3.58 ms
^C
--- 192.168.109.129 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 0.534/2.059/3.584/1.525 ms
toto@Debian12:~$
```

Ping depuis la VMMaster

```
toto@Debian12:~$ ping 192.168.109.128
PING 192.168.109.128 (192.168.109.128) 56(84) bytes of data.
64 bytes from 192.168.109.128: icmp_seq=1 ttl=128 time=0.894 ms
64 bytes from 192.168.109.128: icmp_seq=2 ttl=128 time=1.86 ms
^C
--- 192.168.109.128 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.894/1.378/1.863/0.484 ms
toto@Debian12:~$ ping 192.168.109.130
PING 192.168.109.130 (192.168.109.130) 56(84) bytes of data.
64 bytes from 192.168.109.130: icmp_seq=1 ttl=64 time=5.02 ms
64 bytes from 192.168.109.130: icmp_seq=2 ttl=64 time=1.76 ms
64 bytes from 192.168.109.130: icmp_seq=3 ttl=64 time=1.12 ms
^C
--- 192.168.109.130 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.115/2.631/5.019/1.708 ms
toto@Debian12:~$
```

Ping depuis la VMClient

```
C:\Users\baye>ping 192.168.109.129

Envoi d'une requête 'Ping' 192.168.109.129 avec 32 octets de données :
Réponse de 192.168.109.129 : octets=32 temps=3 ms TTL=64
Réponse de 192.168.109.129 : octets=32 temps=1 ms TTL=64
Réponse de 192.168.109.129 : octets=32 temps=2 ms TTL=64

Statistiques Ping pour 192.168.109.129:
    Paquets : envoyés = 3, reçus = 3, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 1ms, Maximum = 3ms, Moyenne = 2ms
Ctrl+C
^C
C:\Users\baye>ping 192.168.109.130

Envoi d'une requête 'Ping' 192.168.109.130 avec 32 octets de données :
Réponse de 192.168.109.130 : octets=32 temps=2 ms TTL=64
Réponse de 192.168.109.130 : octets=32 temps=2 ms TTL=64
Réponse de 192.168.109.130 : octets=32 temps=2 ms TTL=64

Statistiques Ping pour 192.168.109.130:
    Paquets : envoyés = 3, reçus = 3, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 2ms, Maximum = 2ms, Moyenne = 2ms
Ctrl+C
```

Installation sur la machine Master

Après la mise à jour du système, les paquets nécessaires sont installés :

```
apt update
```

```
apt upgrade -y
```

```
apt install -y python3 python3-venv python3-pip mariadb-server
```

Le service MariaDB est ensuite démarré et activé :

```
systemctl start mariadb
```

```
systemctl enable mariadb
```

Création de l'utilisateur MariaDB

Après connexion au serveur MariaDB :

```
mysql -u root -p
```

Un utilisateur spécifique est créé avec la commande :

```
CREATE USER 'saeuser'@'localhost' IDENTIFIED BY 'sae';
```

Attribution des privilèges

Les privilèges sont accordés uniquement sur la base de données du projet :

```
GRANT ALL PRIVILEGES ON sae302.* TO 'saeuser'@'localhost';
```

```
FLUSH PRIVILEGES;
```

Une base de données dédiée au projet est créée afin de stocker les informations des routeurs :

```
CREATE DATABASE sae302;
```

```
USE sae302;
```

```
CREATE TABLE routeurs (
```

```
    id VARCHAR(10),
```

```
    ip VARCHAR(50),
```

```
    port INT,
```

```
    cle_publique TEXT
```

);

Pour utiliser la base de données saisir la commande : **USE sae302**

Pour visualiser la table routeurs utiliser la commande :

SELECT* FROM routeurs ;

```
MariaDB [sae302]> SELECT* FROM routeurs;
+-----+-----+-----+-----+
| id | ip           | port | cle_publique |
+-----+-----+-----+-----+
| R1 | 192.168.109.130 | 1800 | 65537,2319557 |
| R2 | 192.168.109.130 | 1801 | 65537,5570837 |
| R3 | 192.168.109.130 | 1802 | 65537,6614551 |
| R4 | 192.168.109.130 | 1803 | 65537,9092959 |
+-----+-----+-----+-----+
4 rows in set (0.001 sec)

MariaDB [sae302]>
```

Un environnement virtuel Python est ensuite mis en place afin d'installer le connecteur MariaDB sans perturber le système :

python3 -m venv venv

source venv/bin/activate

pip install mariadb

Le serveur Master est lancé à l'aide de la commande suivante :

python3 master.py

Une fois lancé, le Master affiche les messages indiquant que les serveurs clients et routeurs sont actifs, vous aurez ce résultat ci-dessous

```
(venv) root@Debian12:/home/toto/Downloads# python3 master.py
[MASTER] Démarrage du master (mode serveur)...
[16:13:01] Serveur routeurs actif
[16:13:01] Serveur clients actif
```

Installation sur la machine Routeurs

Sur la machine dédiée aux routeurs, les dépendances nécessaires sont installées :

apt update

apt install -y python3 python3-pip

pip3 install sympy

Chaque routeur est ensuite lancé dans un terminal distinct, avec un identifiant et un port spécifique :

python3 router.py R1 1800

python3 router.py R2 1801

python3 router.py R3 1802

python3 router.py R4 1803

Chaque routeur affiche un message confirmant qu'il est actif et enregistré auprès du Master.

```
root@Debian12:/home/toto/Downloads# python3 router.py R1 1800
[R1] OK
[R1] Routeur actif sur le port 1800
```

Du côté du master le routeur sera enregistré avec ce message

```
{venv} root@Debian12:~# cd /home/toto/Downloads/
{venv} root@Debian12:/home/toto/Downloads# python3 master.py
[MASTER] Démarrage du master (mode serveur)...
[16:13:01] Serveur routeurs actif
[16:13:01] Serveur clients actif
[16:13:53] Routeur connecté depuis ('192.168.109.130', 60452)
[16:13:53] Routeur R1 enregistré
[16:24:19] Routeur connecté depuis ('192.168.109.130', 33518)
[16:24:19] Routeur R1 enregistré
[16:25:19] Routeur connecté depuis ('192.168.109.130', 37732)
[16:25:19] Routeur R2 enregistré
[16:25:47] Routeur connecté depuis ('192.168.109.130', 53226)
[16:25:47] Routeur R3 enregistré
[16:26:24] Routeur connecté depuis ('192.168.109.130', 33190)
[16:26:24] Routeur R4 enregistré
```

Remarque : Au minimum trois routeurs doivent être actifs pour permettre l'envoi d'un message.

Installation sur la machine Clients

Sur la machine Windows, Python est installé ainsi que la bibliothèque PyQt5 :

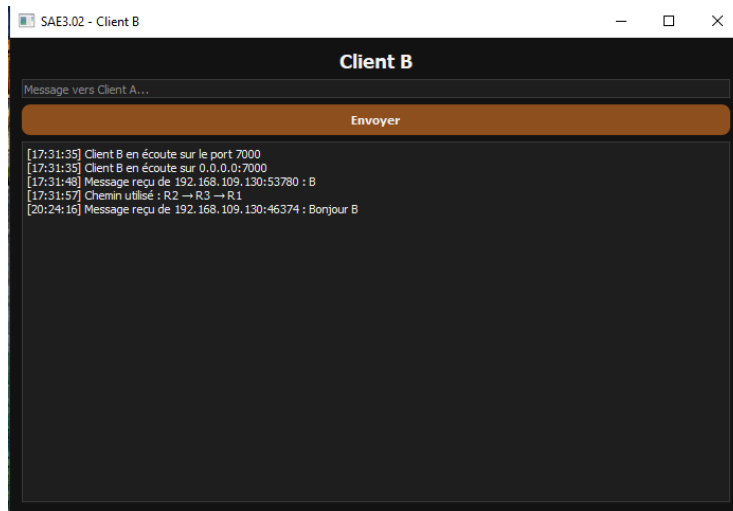
pip install PyQt5

Les interfaces graphiques sont ensuite lancées :

python clientA_gui.py

python clientB_gui.py

Deux fenêtres graphiques apparaissent, correspondant au Client A et au Client B.



Guide d'utilisation

Envoi d'un message du Client A vers le Client B

Pour envoyer un message :

- Le Master doit être lancé ;
- Au moins trois routeurs doivent être actifs ;
- Les interfaces Client A et Client B doivent être ouvertes.

L'utilisateur saisit un message dans le champ prévu à cet effet dans Client A, puis clique sur Envoyer.

Le Client A affiche alors :

- Le chemin utilisé, composé de routeurs choisis aléatoirement visible dans les logs de l'interface du client ;
- Un message confirmant l'envoi via le réseau oignon.

Le Client B reçoit le message final en clair.

Envoi d'un message du Client B vers le Client A

Le fonctionnement est identique dans le sens inverse.

Le Client B chiffre le message, construit un oignon, et le transmet via les routeurs jusqu'au Client A.

Résultats attendus

Lors des tests, le projet doit permettre :

- Une communication bidirectionnelle entre les clients ;
- Un routage aléatoire à chaque envoi ;
- Un chiffrement et déchiffrement progressif par les routeurs ;
- Une interface graphique non bloquante.

Ces éléments permettent de valider la conformité du projet avec le cahier des charges.

Remarques importantes

Les ports doivent être libres avant lancement ;

Le Master doit toujours être lancé avant les routeurs ;

Les routeurs doivent être lancés avant les clients ;

En cas d'erreur de port déjà utilisé, un redémarrage du processus Python est conseillé.